1 Observation Model Physics: Antenna, Attitude, Relativity, and Range Corrections

The corrections in this document are the *deterministic* part of the observation model that lies between pure geometry and the atmosphere on one side, and the streamed SSR overlay on the other. The Foundations observation equations the undifferenced code equation and the undifferenced phase equation introduce the satellite and receiver hardware biases B_s^i, b_s^i, $B_{r,s}$, $b_{r,s}$ *abstractly*, and the observation-geometry section carries the geometric range ρ_r^i and its linearisation; this document supplies every *other* deterministic term that maps the raw measurement to the modelled range: the antenna phase-centre geometry, the satellite attitude frame on which it depends, the carrier-phase wind-up, the relativistic clock and propagation corrections, the Sagnac (Earth-rotation) frame correction, and the deterministic realisation of the bias terms through broadcast group delays and the precise `.BIA` product. The streamed SSR overlay that refines the *same* terms in real time is the subject of the SSR corrections document; the two mechanisms are related but distinct, and the differences (frames, sign conventions, gating) are flagged where they matter.

Every term below is assembled into a single modelled observable in `model/PppModel.cpp` (`PppModel::computeBaseModeledValue`, the single shared helper called by both the iono-free-pair and the uncombined observation paths), summed onto the geometric range that `model/ModelGeometry.cpp` produces. The dominant structure of that sum, for the code observable, is

$$\hat{P}_{r,s}^i = \rho_r^i \;-\; c\,\mathrm{d}t^i \;-\; c\,\delta t_{\mathrm{rel}} \;+\; T_r^i \;+\; A_{r,s}^i \;+\; B_{\mathrm{grp}} \;+\; S_r^i \;+\; d_{\mathrm{atm}} \;+\; d_{\mathrm{tide}}, \tag{1}$$

where $A_{r,s}^i$ is the total antenna correction (Section 1.2), δt_{rel} the periodic relativistic clock term (Section 1.4), S_r^i the Shapiro delay (Section 1.5), B_{grp} the deterministic code/phase bias (Section 1.7), and d_{atm}, d_{tide} the loading/site-displacement terms documented in the site-displacement document. The Sagnac correction (Section 1.6) is folded into ρ_r^i itself, not added separately, for reasons made precise below. The signs in (1) are taken verbatim from the return statement of `PppModel::computeBaseModeledValue` (`model/PppModel.cpp`): the satellite clock and relativistic terms *subtract*, the antenna, Shapiro, loading and tide terms *add*.

1.1 Satellite attitude: the body frame

The satellite antenna phase-centre offset, the antenna phase-centre variation, and the carrier wind-up are all defined in the *spacecraft body frame*, so that frame must be constructed first. The engine uses the IGS/ANTEX convention: a right-handed orthonormal triad $(\hat{\boldsymbol{x}}_i, \hat{\boldsymbol{y}}_i, \hat{\boldsymbol{k}}_i)$ with $+Z$ toward the Earth's centre (nadir / antenna boresight), $+X$ in the Sun–satellite–Earth plane pointing toward the Sun, and $+Y$ completing the triad along the solar-panel (cross-track) axis (`model/SatAttitude.cpp`, `model/Antenna.cpp`).

Nominal yaw-steering. With $\boldsymbol{r}^i$ the satellite ECEF position and $\hat{\boldsymbol{e}}_\odot = (\boldsymbol{r}_\odot - \boldsymbol{r}^i)/\|\boldsymbol{r}_\odot - \boldsymbol{r}^i\|$ the unit direction to the Sun, the nadir axis and the cross-track axis are

$$\hat{\boldsymbol{k}}_i = -\frac{\boldsymbol{r}^i}{\|\boldsymbol{r}^i\|}, \qquad \hat{\boldsymbol{y}}_i = \frac{\hat{\boldsymbol{k}}_i \times \hat{\boldsymbol{e}}_\odot}{\|\hat{\boldsymbol{k}}_i \times \hat{\boldsymbol{e}}_\odot\|}, \qquad \hat{\boldsymbol{x}}_i = \hat{\boldsymbol{y}}_i \times \hat{\boldsymbol{k}}_i, \tag{2}$$

the exact operand ordering of `SatAttitude::compute` (`model/SatAttitude.cpp`), to which the antenna path delegates (`Antenna::getAntennaCorrection`) rather than rebuilding the frame inline;

the one genuine inline duplicate of this construction lives in the wind-up path and is flagged in Section 1.3 below. Forming $\hat{\boldsymbol{y}}_i$ first and then $\hat{\boldsymbol{x}}_i = \hat{\boldsymbol{y}}_i \times \hat{\boldsymbol{k}}_i$ is algebraically the Gram–Schmidt orthogonalisation of the Sun direction against nadir: by the $\boldsymbol{a} \times (\boldsymbol{b} \times \boldsymbol{c})$ identity,

$$\hat{\boldsymbol{x}}_i = \hat{\boldsymbol{y}}_i \times \hat{\boldsymbol{k}}_i = (\hat{\boldsymbol{k}}_i \times \hat{\boldsymbol{e}}_\odot) \times \hat{\boldsymbol{k}}_i = \hat{\boldsymbol{e}}_\odot - \hat{\boldsymbol{k}}_i\,(\hat{\boldsymbol{k}}_i \cdot \hat{\boldsymbol{e}}_\odot), \tag{3}$$

i.e. $\hat{\boldsymbol{x}}_i$ is the component of $\hat{\boldsymbol{e}}_\odot$ orthogonal to nadir, normalised — the $+X$ axis points half-toward the Sun, as required by the ANTEX convention. The body-frame matrix collecting the three columns,

$$\boldsymbol{F} = \big[\, \hat{\boldsymbol{x}}_i \;\; \hat{\boldsymbol{y}}_i \;\; \hat{\boldsymbol{k}}_i \,\big], \tag{4}$$

is orthonormal, so its inverse is its transpose — the property exploited below to rotate a body-frame offset into ECEF.

Sun elevation and the mode switch. For non-nominal attitude (eclipse seasons, the noon/midnight yaw turn) the relevant geometric quantity is the Sun's elevation above the orbital plane, β. With the orbit angular-momentum (normal) unit vector $\hat{\boldsymbol{n}} = (\boldsymbol{r}^i \times \dot{\boldsymbol{r}}^i)/\|\boldsymbol{r}^i \times \dot{\boldsymbol{r}}^i\|$,

$$\boxed{\beta = \arcsin\!\big(\hat{\boldsymbol{n}} \cdot \hat{\boldsymbol{e}}_\odot\big), \qquad \hat{\boldsymbol{n}} = \frac{\boldsymbol{r}^i \times \dot{\boldsymbol{r}}^i}{\|\boldsymbol{r}^i \times \dot{\boldsymbol{r}}^i\|},} \tag{5}$$

signed by the projection of the Sun direction onto the $+(\boldsymbol{r}^i \times \dot{\boldsymbol{r}}^i)$ normal (`SatAttitude::computeBeta`, `model/SatAttitude.cpp`; the sign is noted in-code as unvalidated against an external attitude product). The mode selector (`SatAttitude::determineMode`) keeps GPS, GLONASS and Galileo in yaw-steering unconditionally, and switches BeiDou and QZSS to orbit-normal attitude below a constellation-specific β threshold (4.0° BeiDou, 20.0° QZSS, `model/SatAttitude.cpp`). In orbit-normal mode the $+Y$ axis is pinned to the orbit normal, $\hat{\boldsymbol{y}}_i = \hat{\boldsymbol{n}}$, and the triad is re-orthogonalised $\hat{\boldsymbol{x}}_i = \hat{\boldsymbol{y}}_i \times \hat{\boldsymbol{k}}_i$, $\hat{\boldsymbol{y}}_i = \hat{\boldsymbol{k}}_i \times \hat{\boldsymbol{x}}_i$. The legacy default path additionally forces the hard-coded BeiDou-2 GEO PRN list (C01–C05, C59, C60) into orbit-normal mode irrespective of β; the β-driven table is active only when `enableOrbitNormalAttitude()` is set, and is constructed so the flag-off result is bit-identical to the historical inline frame (`model/Antenna.cpp`, `model/SatAttitude.cpp`).

Eclipse gating. The body frame above presumes a known yaw angle, which is exactly what fails during Earth-shadow passes: the actual spacecraft yaw during eclipse and the post-shadow recovery is block-specific and not modelled here, so the antenna PCO/PCV and the wind-up are unreliable for an eclipsing satellite. When the user opts in (`processing.exclude_eclipsing_satellites`), the satellite is dropped at the geometry stage by a *cylindrical umbra* test (`Planetary::isSatelliteEclipsing`, `model/Planetary.cpp`, called from `model/ModelGeometry.cpp`). With $\hat{\boldsymbol{u}}_\odot = \boldsymbol{r}_\odot/\|\boldsymbol{r}_\odot\|$ the Earth-to-Sun direction, the satellite's along-axis and perpendicular distances are $s_\parallel = \boldsymbol{r}^i \cdot \hat{\boldsymbol{u}}_\odot$ and $\boldsymbol{s}_\perp = \boldsymbol{r}^i - s_\parallel\,\hat{\boldsymbol{u}}_\odot$, and the predicate is

$$\text{eclipsed} \iff \big(s_\parallel < 0\big) \;\wedge\; \big(\|\boldsymbol{s}_\perp\| < \kappa\, R_\oplus\big), \tag{6}$$

i.e. the satellite is on the anti-Sun side *and* inside the shadow cylinder of radius $\kappa R_\oplus$, with $R_\oplus = 6\,371\,230$ m (`common/Constants.hpp`) and κ the user shadow scale (`processing.eclipse_shadow_scale`, default unity; cited rather than reproduced as a tuning knob). This is a deliberately conservative geometric umbra, not a photometric penumbra model.

1.2 Antenna phase-centre corrections

The range measurement is realised between the satellite and receiver *antenna phase centres*, but the estimated states refer to the satellite centre of mass and the receiver antenna reference point (ARP): the ANTEX receiver PCO/PCV below are ARP-referenced, so the position state must be too. The antenna correction $A^i_{r,s}$ reconciles the two, summing four geometric terms plus the wind-up:

$$\boxed{A^i_{r,s} \;=\; -\,\delta^i_{\mathrm{PCO}} \;+\; \delta^i_{\mathrm{PCV}} \;+\; \delta^r_{\mathrm{PCO}} \;+\; \delta^r_{\mathrm{PCV}} \;+\; \varphi_{\mathrm{wu}},} \tag{7}$$

the exact accumulation of `Antenna::getAntennaCorrection` (`model/Antenna.cpp`). The line-of-sight used throughout is the *satellite-to-receiver* unit vector, $\boldsymbol{e} = (\boldsymbol{r}_r - \boldsymbol{r}^i)/\|\boldsymbol{r}_r - \boldsymbol{r}^i\|$ (`Antenna.cpp`; note this is the negative of the receiver-to-satellite $\boldsymbol{e}^i_r$ of the geometry section in its line-of-sight definition). The sign conventions are derived once and reused for each term below.

Which coordinate is the ARP. The ARP frame asserted above holds for every position the engine derives from a RINEX *header*: the marker coordinate is reduced to the ARP through the header eccentricity before it seeds the state (`Coordinates::markerToArp`, packaged as `RinexParser::getArpPositionXyz`; the marker/ARP section of the Coordinate Frames document). A coordinate supplied directly in the configuration is *not* reduced — it is used at face value, on the policy that the user owns its reference point. That exception is load-bearing in the network producer, whose station coordinates come from the configuration (`StationConfig::position_ecef`) and are held fixed, their position columns dropped from the normal equations: a monument-referenced coordinate there is not absorbed by a position state but survives into the residuals, and so into the estimated clocks, biases, and ionosphere.

Satellite PCO (subtracted). The satellite phase-centre offset $\boldsymbol{p}^i_{\mathrm{body}}$ is published in the body frame (millimetres in ANTEX). It is rotated to ECEF by the body-frame matrix — because $\boldsymbol{F}$ collects the body axes as *columns*, $\boldsymbol{F}$ itself maps body $\to$ ECEF (and the orthonormal $\boldsymbol{F}^{\mathrm{T}} = \boldsymbol{F}^{-1}$ maps ECEF $\to$ body) — and projected onto the LOS:

$$\delta^i_{\mathrm{PCO}} = \big(\boldsymbol{F}\,\boldsymbol{p}^i_{\mathrm{body}}\big)\cdot\boldsymbol{e}. \tag{8}$$

Because the antenna phase centre sits *nearer the receiver* than the centre of mass, the true range is shorter than the CoM range; the correction is therefore *subtracted* in (7) (`correction -= sat_pco_corr`, `model/Antenna.cpp`). One real-time exception omits this term entirely: when an applied SSR orbit correction is itself APC-referenced (the corrected orbit already points at the phase centre), the `skip_satellite_pco` path of `getAntennaCorrection` drops the satellite-PCO range term to avoid double-counting it, gated by `realtime.honor_ssr_reference_point` and default-off; the SSR reference-point dialect that drives it is documented with the SSR corrections document.

Satellite PCV (added). The satellite phase-centre variation is a function of the *nadir angle* — the angle between the boresight $\hat{\boldsymbol{k}}_i$ and the line of sight — and of the azimuth in the body frame. With $\cos\theta_{\mathrm{nad}} = \hat{\boldsymbol{k}}_i \cdot \boldsymbol{e}$,

$$\theta_{\mathrm{nad}} = \arccos\big(\hat{\boldsymbol{k}}_i \cdot \boldsymbol{e}\big), \qquad \alpha_{\mathrm{body}} = \mathrm{atan2}\big((\boldsymbol{F}^{\mathrm{T}}\boldsymbol{e})_y,\,(\boldsymbol{F}^{\mathrm{T}}\boldsymbol{e})_x\big), \tag{9}$$

and the PCV value is read by bilinear interpolation of the ANTEX grid over (zenith/nadir ring, azimuth sector), the look-up argument being $\theta_{\mathrm{nad}} - \texttt{zen1}$ in grid units (`Antenna::interpolatePcv`, `model/Antenna.cpp`). The satellite PCV is *added* to the range correction.

Receiver PCO (added). The receiver PCO is published as a North/East/Up (NEU) triplet relative to the ARP. It is reordered to ENU, rotated to ECEF by the local ENU→ECEF rotation $\boldsymbol{R}_{\text{enu}}(\phi,\lambda)$ at the receiver geodetic latitude/longitude (`Coordinates::enuToEcefMatrix`), and projected on the LOS:

$$\delta^r_{\text{PCO}} = \Big(\boldsymbol{R}_{\text{enu}}(\phi,\lambda)\,\big[\,p_E\;\;p_N\;\;p_U\,\big]^{\mathrm{T}}\Big)\cdot\boldsymbol{e}, \qquad \big[\,p_E\,p_N\,p_U\,\big] = \big[\,p_{\text{NEU},2}\;\;p_{\text{NEU},1}\;\;p_{\text{NEU},3}\,\big], \tag{10}$$

where the index swap $(N,E,U)\to(E,N,U)$ is explicit in `model/Antenna.cpp`. It is *added* to the correction.

Receiver PCV (added). The receiver PCV is indexed by *zenith angle* $z = 90°-\text{el}$ (and, optionally, azimuth corrected for the antenna mounting orientation when `enableRxAntennaOrientation()` is set), with the same bilinear interpolation over $z-$ `zen1` (`model/Antenna.cpp`). It is *added* to the correction.

ATX frequency fallback. ANTEX entries frequently omit the newest carriers (L5/E5a/B2a on older spacecraft, and many receiver antennas published before triple-frequency tracking). Rather than dropping the antenna correction to zero — which leaves a several-centimetre vertical bias — the engine walks a *nearest-carrier* substitution chain when the requested ANTEX frequency key is absent (`pickAtxFrequencyKey`, `model/Antenna.cpp`). The chains are ordered by carrier-frequency proximity, e.g. for GPS `G05` → {`G02`, `G01`} and for Galileo `E05` → {`E08`, `E07`, `E06`, `E01`}, since cross-frequency PCO drift on one physical antenna is typically below a centimetre. The PCO and PCV chains are walked *independently* (an antenna may carry the PCO but not the PCV at a given key). For the *satellite* antenna only the intra-system chain is permitted — borrowing another constellation's spacecraft entry would be meaningless. For the *receiver* antenna, after the intra-system chain is exhausted and only when `enable_receiver_cross_system_atx_fallback` is set, a *cross-system* nearest-GPS-band chain is allowed (`pickReceiverAtxFrequencyKey`), because the receiver phase pattern is a function of frequency on shared hardware: the load-bearing mappings are exact-carrier matches (`E01` → `G01`, `E05` → `G05` at 1575.42 and 1176.45 MHz), the remainder ∼10–70 MHz approximations (`model/Antenna.cpp`). Each substitution raises a coverage flag and emits a fallback log line; this is the same graceful-fallback policy used for biases rather than excluding the measurement.

1.3 Carrier-phase wind-up

A right-hand-circularly-polarised GNSS signal accrues a phase shift equal to the relative geometric rotation between the transmitting and receiving antenna dipoles as either rotates about the line of sight [2]. This wind-up is identical on every signal in cycles (hence a range term proportional to wavelength) and is computed in `GnssCorrections::computePhaseWindUp` (`corrections/GnssCorrections.cpp`). With $\boldsymbol{k}$ the unit line-of-sight, $(\hat{\boldsymbol{x}}_i,\hat{\boldsymbol{y}}_i)$ the satellite body $\pm X,+Y$ axes and $(\boldsymbol{x}_r,\boldsymbol{y}_r)$ the receiver local North/East axes, the *effective dipoles* are

$$\boldsymbol{D}_i = \hat{\boldsymbol{x}}_i - \boldsymbol{k}\,(\boldsymbol{k}\cdot\hat{\boldsymbol{x}}_i) + \boldsymbol{k}\times\hat{\boldsymbol{y}}_i, \qquad \boldsymbol{D}_r = \boldsymbol{x}_r - \boldsymbol{k}\,(\boldsymbol{k}\cdot\boldsymbol{x}_r) + \boldsymbol{k}\times\boldsymbol{y}_r, \tag{11}$$

the standard Wu construction — each dipole is the antenna X axis projected into the plane orthogonal to $\boldsymbol{k}$, plus the $\boldsymbol{k}\times Y$ quadrature term (`corrections/GnssCorrections.cpp`). The receiver North and East axes are built directly from the geodetic latitude ϕ and longitude λ, $\boldsymbol{x}_r = (-\sin\phi\cos\lambda,\,-\sin\phi\sin\lambda,\,\cos\phi)$ and $\boldsymbol{y}_r = (-\sin\lambda,\,\cos\lambda,\,0)$ (`GnssCorrections.cpp`). The

signed wind-up angle is the atan2 of the cross- and dot-products of the unit dipoles, which recovers the full $(-\pi, \pi]$ range and avoids the information loss of an arccos near 0 and π:

$$\boxed{\Delta\varphi = \operatorname{atan2}\Big(\boldsymbol{k}\cdot\big(\hat{\boldsymbol{D}}_i \times \hat{\boldsymbol{D}}_r\big),\ \hat{\boldsymbol{D}}_i \cdot \hat{\boldsymbol{D}}_r\Big), \qquad \varphi_{\mathrm{wu}} = \Delta\varphi\,\frac{\lambda_s}{2\pi},} \tag{12}$$

with $\hat{\boldsymbol{D}}_i, \hat{\boldsymbol{D}}_r$ the normalised dipoles and λ_s the signal wavelength (`corrections/GnssCorrections.cpp`); the $\boldsymbol{k}\cdot(\hat{\boldsymbol{D}}_i \times \hat{\boldsymbol{D}}_r)$ argument fixes the sign of the rotation about the line of sight. The result enters (7) as φ_{wu} on phase observables only (the code branch passes zero wavelength).

Fractional vs. accumulated modes. The raw atan2 returns the wind-up wrapped to a single cycle. In the default *fractional* mode this wrapped angle is used directly: the float ambiguity $N^i_{r,s}$ absorbs any integer-cycle component, so wrapping is harmless. For integer ambiguity resolution the ambiguity must stay a constant integer across the arc, so the engine offers an *accumulated* (unwrapped) mode (`config.accumulatePhaseWindUp()`) that unwraps each epoch relative to the previous stored angle, $\Delta\varphi_t = \Delta\varphi_{t-1} + \mathrm{wrap}_{2\pi}(\Delta\varphi_t - \Delta\varphi_{t-1})$, provided the inter-epoch gap is below the 300 s continuity threshold; a larger gap (or a backward time step) restarts the accumulator from the raw angle (`corrections/GnssCorrections.cpp`). This 300 s reset is the wind-up analogue of an ambiguity-arc boundary: a gap long enough to invalidate the unwrap reference is treated as a fresh arc.

Known frame duplication. The satellite body frame feeding the wind-up dipole $\boldsymbol{D}_i$ (11) is built *inline* inside `computePhaseWindUp` (`corrections/GnssCorrections.cpp`) rather than calling the shared `model/SatAttitude.cpp` helper used by the antenna path (2). The two constructions are deliberately bit-for-bit identical (same operand ordering, same BeiDou-2 GEO orbit-normal special case, same velocity fallback), and both source files note the duplication explicitly. It is flagged here because the invariant is a *maintenance* obligation: a change to one frame must be mirrored in the other or the antenna PCV and the wind-up would silently diverge on the same satellite.

1.4 Periodic relativistic clock correction

A clock on an eccentric orbit runs at a time-varying rate relative to a clock on the geoid; the constant and secular parts are absorbed into the broadcast/precise clock polynomial, but the *periodic* part driven by orbital eccentricity must be applied separately. The standard expression [1], in `GnssCorrections::computeRelativisticCorrection` (`corrections/GnssCorrections.cpp`), is

$$\boxed{\delta t_{\mathrm{rel}} \;=\; -\,\frac{2\,(\boldsymbol{r}^i \cdot \dot{\boldsymbol{r}}^i)}{c^2} \quad [\mathrm{s}], \qquad \text{range term} \;= -\,c\,\delta t_{\mathrm{rel}},} \tag{13}$$

i.e. $-2\,(\boldsymbol{r}\cdot\boldsymbol{v})/c^2$ seconds, applied to the range by the $-c\,\delta t_{\mathrm{rel}}$ term of the assembly (1) (`model/PppModel.cpp`). Two points are worth making precise. First, the dot product $\boldsymbol{r}^i \cdot \dot{\boldsymbol{r}}^i$ is *invariant* to whether the position/velocity pair is expressed in the inertial or the co-rotating ECEF frame: the ECEF velocity differs from the inertial velocity by the frame-rotation term $\dot{\boldsymbol{r}}^i_{\mathrm{ecef}} = \dot{\boldsymbol{r}}^i_{\mathrm{inertial}} - \boldsymbol{\omega}_\oplus \times \boldsymbol{r}^i$, and $\boldsymbol{r}^i \cdot (\boldsymbol{\omega}_\oplus \times \boldsymbol{r}^i) = 0$ because the cross-product is orthogonal to $\boldsymbol{r}^i$; so the same scalar results from either velocity, and the broadcast-ephemeris ECEF velocity may be used directly. Second, the periodic term is computed *here*, separately, because the IGS precise CLK product by convention carries only the constant-plus-drift clock and excludes the periodic eccentricity term — applying (13) restores the missing piece consistently for both precise and broadcast clock sources.

1.5 Shapiro (gravitational-delay) correction

The signal path bends and slows in the Earth's gravitational potential; the resulting Shapiro delay is a few centimetres [1]. With $r_i = \|\boldsymbol{r}^i\|$, $r_r = \|\boldsymbol{r}_r\|$ the geocentric radii and $r_{ir} = \|\boldsymbol{r}^i - \boldsymbol{r}_r\|$ the satellite–receiver distance, the delay (in metres) is

$$\boxed{S_r^i \;=\; \frac{2\,GM}{c^2}\,\ln\frac{r_i + r_r + r_{ir}}{r_i + r_r - r_{ir}},} \tag{14}$$

with $GM = 3.986005 \times 10^{14}\,\mathrm{m}^3/\mathrm{s}^2$ (`Constants::GRAVITATIONAL_CONSTANT`, `common/Constants.hpp`) and the formula verbatim at `GnssCorrections.cpp`. Because $r_{ir} < r_i + r_r$ the logarithm is positive, so $S_r^i > 0$ and the term is *added* to the modelled range in (1) (`model/PppModel.cpp`). Its magnitude is ∼1–2 cm for a typical MEO geometry.

1.6 Sagnac (Earth-rotation) correction

The pseudorange is formed between the satellite position at *signal emission* and the receiver position at *signal reception*, but both are needed in a single, consistent ECEF frame. During the ∼70 ms transit the ECEF frame itself rotates with the Earth, so the emission-time satellite coordinates must be rotated forward into the reception-time frame. The engine applies the *rigorous* rotation, not a scalar approximation: with the geometric time-of-flight $\tau = \|\boldsymbol{r}^i_{\mathrm{em}} - \boldsymbol{r}_r\|/c$ and the Earth rotation rate $\omega_\oplus = 7.2921151467 \times 10^{-5}\,\mathrm{rad/s}$ (`common/Constants.hpp`), the emission-time position $\boldsymbol{r}^i_{\mathrm{em}}$ is rotated by $\theta = \omega_\oplus \tau$ about the Z axis,

$$\boxed{\boldsymbol{r}^i = \boldsymbol{R}_z(\omega_\oplus \tau)\,\boldsymbol{r}^i_{\mathrm{em}}, \qquad \boldsymbol{R}_z(\theta) = \begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix},} \tag{15}$$

after which $\rho_r^i = \|\boldsymbol{r}^i - \boldsymbol{r}_r\|$ and the line of sight are computed from the rotated $\boldsymbol{r}^i$ (`model/ModelGeometry.cpp`, the `rotateAndComputeTopo` lambda). The Sagnac correction is thus *folded into the geometric range itself* rather than added as a separate term — which is why (1) carries no explicit Sagnac entry. An equivalent scalar form, $(\omega_\oplus/c)\,(x_i y_r - y_i x_r)$, exists in the codebase as a *diagnostic* only (`GnssCorrections::computeSagnacCorrection`, `corrections/GnssCorrections.cpp`); it is the first-order expansion of the rigorous rotation and is not on the modelling path. The Orbit and Clock Product Evaluation document cross-references *here* for the Sagnac frame rotation; the derivation is kept in this document. The same $\boldsymbol{R}_z(\omega(t_{\mathrm{node}} - t_{\mathrm{query}}))$ rotation is reused at SP3 interpolation time to pre-rotate ephemeris nodes into the query-epoch frame (`model/ModelGeometry.cpp`), a numerical-conditioning use of the identical rotation documented with the products.

1.7 Group delays and inter-frequency biases

The abstract bias terms of the Foundations observation equations — the satellite code/phase biases B_s^i, b_s^i and the receiver biases $B_{r,s}$, $b_{r,s}$ — are realised deterministically by two products, which this section ties to the equation symbols.

Broadcast group delay. The broadcast navigation message carries a per-system inter-frequency group delay (GPS TGD, Galileo BGD, BeiDou TGD), referencing each signal's code hardware delay to the ionosphere-free combination used to generate the broadcast clock. In the uncombined parameterisation these are absorbed into the deterministic code bias applied through `model/`

PppModel.cpp's bias term; they are the broadcast-grade realisation of B_s^i, used when no precise bias product is available.

Precise OSB / DCB biases. When a precise bias product is loaded, the deterministic correction is the observable-specific bias (OSB) / differential code bias (DCB) entry for the exact (i,s) pair, time-windowed against its validity interval and returned by `GnssCorrections::getBiasCorrection` (`corrections/GnssCorrections.cpp`). This single scalar realises B_s^i (code) or b_s^i (phase) for that signal:

$$B_{\text{grp}} = \text{bias}(i, s, t), \qquad t_{\text{start}} \le t \le t_{\text{end}}, \tag{16}$$

with the application gated by `enable_code_phase_bias` (`model/PppModel.cpp`). Two exclusions are baked into the bias index (`GnssCorrections::buildBiasIndex`): *zero-valued* code entries (e.g. MGEX placeholder `C2I`/`C6I` for BeiDou) are treated as *absent* so the model does not both apply a null correction and suppress the per-(i,s) estimated bias state, leaving the real tens-of-metres delay unmodelled; and *GLONASS code* entries are likewise treated as missing because the FDMA per-channel receiver delay cannot be captured by a single value-per-(i,s) bias, forcing those signals onto the estimated-state path (`corrections/GnssCorrections.cpp`).

Estimated vs. deterministic biases. Whatever the deterministic bias leaves uncorrected — principally the receiver hardware delay $B_{r,s}$, $b_{r,s}$, and any satellite delay not covered by a product — is handled by the estimator. The receiver biases are common to all satellites on a signal and so collapse into the receiver clock / inter-system bias states or cancel under between-satellite differencing (the differencing section); the per-(i,s) satellite code bias may be *estimated* as a state when no product covers it (`estimation.estimate_sat_biases`, `measurement/MeasurementModel.cpp`). The state-allocation gate that prevents estimating a bias the products already supply — hasBIA(i, s) from the static product, OR'd with hasSsrCB(i, s) from the live stream — is documented with the SSR overlay (the SSR corrections document, **??** there); the present document supplies only the *deterministic* half of that OR.

Relation to the SSR overlay. The streamed SSR code/phase OSB (the SSR corrections document) refines the *same* B_s^i, b_s^i terms in real time, summed onto the predicted observable as `ssr_bias_term` in `measurement/MeasurementModel.cpp` alongside the deterministic B_{grp} here. The two are distinct mechanisms and must not be conflated: the SSR bias is a per-(i,s) scalar like (16), but the SSR *orbit* delta lives in a radial/along/cross (RAC) delta frame (the SSR document's **??**) that is *not* the satellite attitude body frame (4) of this document — the RAC frame is built from the broadcast position and velocity to express an orbit correction, whereas the body frame is built from the Sun direction to orient the antenna. They coincide neither in construction nor in purpose.

1.8 Provenance and assembly

All deterministic corrections converge in `model/PppModel.cpp`: `computeBaseModeledValue`, the single shared helper for both the iono-free-pair and uncombined paths, returns the sum (1), calling `model/ModelGeometry.cpp` for the Sagnac-rotated geometric range (15), `model/Antenna.cpp` for the total antenna correction (7) (which in turn calls `model/SatAttitude.cpp` for the body frame and `corrections/GnssCorrections.cpp` for the wind-up), and `corrections/GnssCorrections.cpp` for the relativistic (13), Shapiro (14) and bias (16) terms. The Sun and Moon ephemeris feeding the attitude frame (2), the Sun-elevation (5), the eclipse predicate (6) and the wind-up (11) is produced by `model/Planetary.cpp` (the rigorous IAU precession/nutation Sun position, cached

per epoch) and is documented in the Coordinate Frames and Time Systems document. Every term is invariant physics; the only user-facing knobs are enable toggles and the eclipse-shadow scale, all cited rather than reproduced because they tune *whether* a correction is applied, never its functional form.

References

[1] J. Sanz Subirana, J. M. Juan Zornoza, and M. Hernández-Pajares. *GNSS Data Processing, Volume I: Fundamentals and Algorithms*. ESA TM-23/I. Noordwijk, The Netherlands: ESA Communications, 2013.

[2] G. Wübbena. "Software developments for geodetic positioning with GPS using TI 4100 code and carrier measurements". In: *Proceedings of the First International Symposium on Precise Positioning with the Global Positioning System*. Rockville, MD, USA, 1985, pp. 403–412.